

Camada de Transporte

TCP: Controle de Congestionamento

Princípios do Controle de Congestionamento

O Problema

Informalmente: “Muitas fontes enviando muitos dados muito rápido para que a rede consiga manipular”

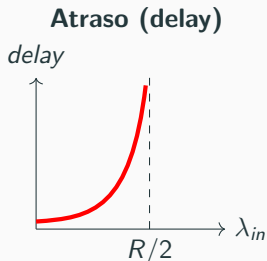
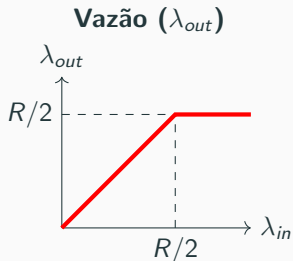
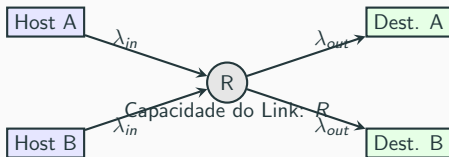
Características Fundamentais:

- É diferente do **controle de fluxo**!
- Considerado um dos 10 problemas principais da Internet

Sinais de Alerta:

- **Perda de pacotes:** Estouro de buffer nos roteadores
- **Longos atrasos:** Filas extensas nos buffers

Cenário I: Vazão e Atraso com Buffers Infinitos

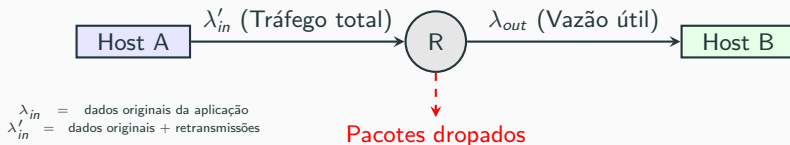


- **Vazão Máxima:** Estabiliza em $R/2$ por conexão
- **Custo do Congestionamento:** O atraso tende ao infinito conforme a carga se aproxima da capacidade total do roteador

Cenário II: Realidade dos Buffers Finitos

Premissas do Modelo:

- Um único roteador com memória limitada (buffers finitos)
- Pacotes podem ser descartados (*dropped*) se o buffer estiver cheio
- O transmissor retransmite pacotes perdidos ou com *timeout*



Consequência da Retransmissão

A carga oferecida à rede (λ'_{in}) agora é sempre maior ou igual à carga da aplicação (λ_{in}), aumentando a pressão sobre os recursos do roteador

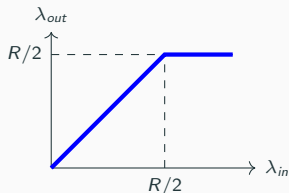
Causas e Custos: Cenário II (Caso Ideal)

Idealização: O transmissor possui “conhecimento perfeito” sobre o estado da rede

Dinâmica de Envio



Vazão (Throughput)



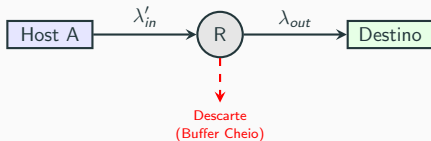
Consequências do Conhecimento Perfeito

- O transmissor envia pacotes **apenas** quando sabe que há espaço livre no buffer do roteador
- **Zero perdas:** Nenhum pacote é descartado (*dropado*)
- **Sem retransmissões:** A carga oferecida à rede é exatamente igual à carga da aplicação ($\lambda_{in} = \lambda'_{in}$)
- A vazão máxima teórica de $R/2$ é alcançada sem qualquer desperdício

Causas e Custos: Cenário II (Perda Conhecida)

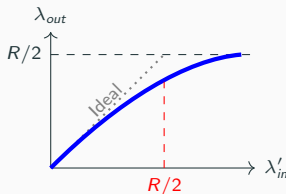
Situação Intermediária: O transmissor sabe exatamente quando um pacote é descartado pelo roteador e retransmite **apenas** esse pacote

Dinâmica com Retransmissão



A carga oferecida (λ'_{in}) agora é a soma dos dados originais mais as retransmissões

Vazão (λ_{out})



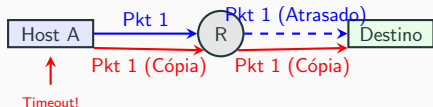
O Primeiro “Custo” do Congestionamento

Para que a vazão útil (λ_{out}) consiga se aproximar do limite da rede ($R/2$), o transmissor é forçado a enviar dados a uma taxa (λ'_{in}) **maior** que a capacidade, para compensar as perdas

Causas e Custos: Cenário II (Timeouts Prematuros)

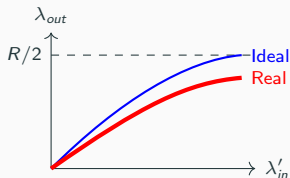
Situação Realista: A rede possui atrasos variáveis. O transmissor sofre um *timeout* prematuro e retransmite pacotes que estão apenas atrasados na fila do roteador

Dinâmica da Duplicação



Roteador encaminha **ambas** as cópias para a frente. Destino recebe o mesmo dado duas vezes

O Impacto na Vazão



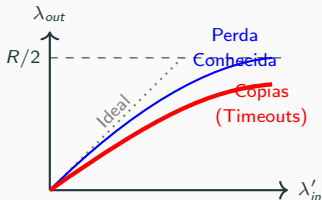
O Segundo “Custo” do Congestionamento

O transmissor realiza trabalho inútil reenviando pacotes. A rede desperdiça sua capacidade de transmissão movendo **cópias desnecessárias** do mesmo pacote, impedindo que a vazão útil máxima seja alcançada

Causas e Custos: Resumo do Cenário II

Síntese dos Custos de Congestionamento

Comparativo de Vazão



Custos Identificados:

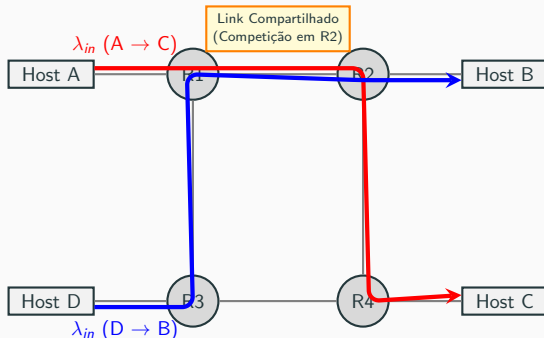
- **Custo 1 (Trabalho Extra):** Para atingir uma determinada vazão útil, o transmissor e a rede precisam trabalhar mais (retransmitir) devido às perdas reais
- **Custo 2 (Desperdício de Banda):** Timeouts prematuros geram retransmissões desnecessárias. O enlace é ocupado transportando múltiplas cópias do mesmo pacote

Consequência Principal

As retransmissões desnecessárias degradam o desempenho. A vazão útil máxima que a rede consegue entregar cai significativamente em relação à sua capacidade teórica

Causas e Custos: Cenário III (Introdução)

Contexto Completo: Quatro transmissores, roteadores com buffers finitos e caminhos com múltiplos saltos cruzados



A Dinâmica dos Saltos:

- O fluxo **Azul** divide o link $R1 \rightarrow R2$ com o fluxo **Vermelho**
- Ambos competem intensamente pelo espaço restrito no buffer do roteador **R2**

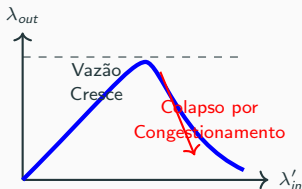
O Problema Inicial:

- O que acontece com a vazão do fluxo **Azul** se o Host A começar a enviar dados (fluxo **Vermelho**) a uma taxa absurdamente alta?

Causas e Custos: Cenário III (O Colapso)

A Consequência da Concorrência Extrema: O que ocorre quando a carga oferecida (λ'_{in}) de um fluxo se torna excessivamente agressiva?

O Gráfico do Colapso



Análise do Comportamento:

- Conforme o tráfego **Vermelho** inunda o roteador R2, ele ocupa todo o buffer disponível
- Os pacotes do fluxo **Azul** que chegam a R2 encontram a fila cheia e são descartados
- A vazão útil (λ_{out}) do fluxo Azul despenca para zero, mesmo com o Host D enviando a toda velocidade

O Terceiro “Custo” (Desperdício Upstream)

Quando um pacote é descartado em um roteador no meio do caminho (ex: R2), **toda** a capacidade de transmissão utilizada nos saltos anteriores (ex: de R3 para R1, e de R1 para R2) foi desperdiçada

Controle de Congestionamento TCP: AIMD

A Abordagem do TCP: Como a rede IP não fornece feedback explícito de congestionamento, o transmissor usa a **perda de pacotes** como sinal para reduzir sua taxa

AIMD (Additive Increase, Multiplicative Decrease):

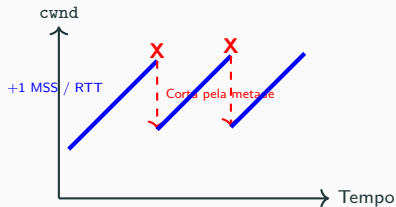
- **Aumento Aditivo:** Aumenta a Janela de Congestionamento (cwnd) em 1 MSS (Tamanho Máximo do Segmento) a cada RTT sem perdas
- **Redução Multiplicativa:** Corta a cwnd pela metade após detectar uma perda

Taxa de Envio

O transmissor limita a taxa baseando-se na cwnd:

$$\text{Taxa} \approx \frac{\text{cwnd}}{\text{RTT}} \text{ bytes/seg}$$

Comportamento “Dente de Serra”

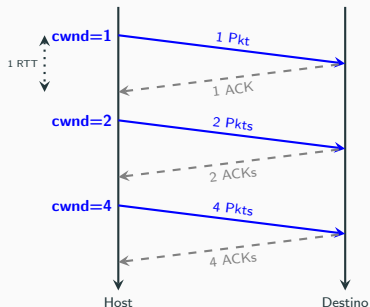


Sondagem de Banda: O TCP aumenta a taxa gradativamente até a rede congestionar, recua drasticamente e tenta novamente

Controle de Congestionamento TCP: Partida Lenta

O Problema Inicial: A conexão TCP começa sem saber a capacidade real da rede. Aumentar a janela linearmente demoraria muito para utilizar a banda disponível

A Mecânica do Slow Start



Como funciona?

- A Janela de Congestionamento (cwnd) inicia com o valor de **1 MSS**
- Para **cada** ACK recebido, a cwnd é incrementada em 1 MSS
- **Efeito Prático:** A taxa de envio dobra a cada RTT

O Paradoxo do Nome

Apesar de se chamar “Partida Lenta” (pois começa do zero absoluto, enviando apenas 1 pacote), o crescimento da taxa é **exponencial**: 1, 2, 4, 8, 16...

Controle de Congestionamento: Reação a Perdas

O **Limiar** ($ssthresh$): O crescimento exponencial não dura para sempre. O limite $ssthresh$ define quando o TCP muda sua estratégia para evitar sobrecarregar a rede

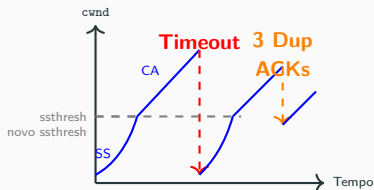
Fases de Crescimento:

- $cwnd < ssthresh$: **Partida Lenta** (Exponencial)
- $cwnd \geq ssthresh$: **Prevenção** (Linear, +1 MSS)

Tipos de Perda e Reação:

- **Timeout (Congestionamento Severo):**
 - $ssthresh \leftarrow cwnd/2$
 - $cwnd \leftarrow 1 \text{ MSS}$ (Zera a janela)
- **3 ACKs Duplicados (Cong. Leve):**
 - $ssthresh \leftarrow cwnd/2$
 - $cwnd \leftarrow ssthresh$ (Corta pela metade)

Dinâmica da Janela ($cwnd$)



Controle de Congestionamento: Tahoe vs. Reno

A Evolução do Protocolo: Historicamente, o TCP evoluiu para lidar de forma mais inteligente com perdas leves (3 ACKs duplicados), diferenciando-as de congestionamentos severos (Timeouts)

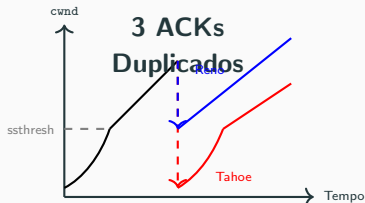
TCP Tahoe (Versão Antiga):

- Não diferenciava o tipo de perda
- Qualquer perda (Timeout ou 3 ACKs Duplicados) resultava na *cwnd* sendo reduzida para **1 MSS**
- Reiniciava a fase de *Slow Start*, o que derrubava a vazão drasticamente

TCP Reno (Versão Moderna):

- Introduziu a **Recuperação Rápida (Fast Recovery)**
- Se houver Timeout: *cwnd* cai para 1 MSS
- Se houver 3 ACKs Dup: A rede ainda está entregando algo. A *cwnd* cai apenas pela metade (*ssthresh*) e retoma em **Prevenção (Linear)**

Comparativo de Desempenho



Controle de Congestionamento: Vazão do TCP

Modelo Macroscópico: Qual é a vazão média (throughput) de uma conexão TCP em estado estacionário, ignorando o *Slow Start*?

Análise do Ciclo AIMD:

- W : Janela (em bytes) no exato momento da perda
- **Vazão Máxima:** W/RTT
- **Vazão Mínima:** $(W/2)/\text{RTT}$

1. Vazão Média Teórica:

Como o crescimento é linear, a média é exata:

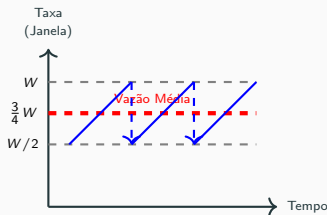
$$\text{Média} = \frac{0.75 \cdot W}{\text{RTT}}$$

2. Em Função da Perda (L):

Considerando a taxa de perda de pacotes da rede:

$$\text{Vazão} \approx \frac{1.22 \cdot \text{MSS}}{\text{RTT} \cdot \sqrt{L}}$$

Vazão ao Longo do Tempo



Controle de Congestionamento: Vazão e Perda

A Equação de Mathis: A modelagem do ciclo AIMD revela como a perda de pacotes (L) afeta drasticamente a vazão do TCP

$$\text{Vazão} \approx \frac{1.22 \cdot \text{MSS}}{\text{RTT} \cdot \sqrt{L}}$$

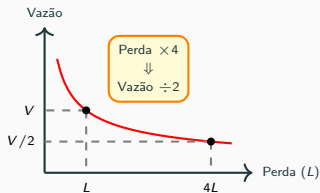
A Origem do 1.22 ($\sqrt{3/2}$):

A área do gráfico em dente de serra (pacotes enviados por ciclo) é $\frac{3}{8} W^2$. Como há 1 perda por ciclo, $L = \frac{8}{3W^2}$. Isolando W e aplicando na equação da média, surge a constante $\sqrt{1.5} \approx 1.22$

O Efeito da Raiz Quadrada:

O RTT limita a banda, mas o L na raiz quadrada no denominador é fatal. Se a rede passa de 1% para 4% de perda (aumento de 4x), a raiz vira 2 e a **vazão cai pela metade**

Impacto da Perda na Vazão



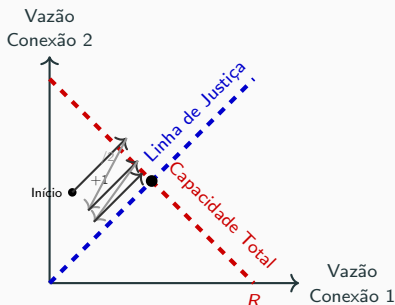
Controle de Congestionamento: Justiça do TCP

A Matemática da Convergência: O AIMD garante que múltiplas conexões TCP concorrentes dividam o link de forma igualitária, mesmo sem comunicação direta

A Dinâmica do AIMD:

- **Aumento Aditivo (+1 MSS):**
Ambas as conexões crescem suas janelas. O estado move-se paralelamente à Linha de Justiça
- **Redução Multiplicativa ($/2$):**
Ambas as conexões cortam suas taxas pela metade ao detectar perda. O estado move-se em direção à origem

Convergência de Duas Conexões



O Processo de Zigue-Zague

A combinação dessas duas ações força o estado a ziguezaguear e convergir para o cruzamento exato da Capacidade Total com a Linha de Justiça

Controle de Congestionamento: Burlando a Justiça

O Mundo Real: A justiça matemática do TCP pressupõe cooperação. Porém, aplicações podem adotar estratégias egoístas para capturar mais banda do que teriam direito

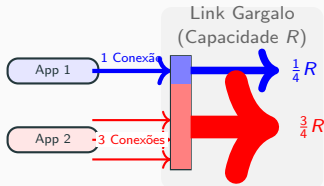
1. O Problema do UDP:

- Aplicações multimídia (streaming, VoIP) frequentemente usam UDP para não sofrerem redução de taxa
- **Sem Controle:** O UDP não possui AIMD. Ele não recua quando há perdas
- **Consequência:** O UDP “atropela” conexões TCP cooperativas, que reduzem suas taxas enquanto o UDP continua inundando o link

2. Conexões TCP Paralelas:

- Nada impede uma aplicação de abrir dezenas de conexões simulando ser “vários usuários”
- **A Matemática:** Se 9 conexões dividem um link R , cada uma recebe $R/9$
- Se um usuário abre **11 conexões novas**, ele passa a dominar $11/20$ da banda, roubando mais da metade do link de gargalo

Dinâmica das Conexões Paralelas



Exemplos Comuns: Navegadores Web (vários objetos) e Gerenciadores de Download paralelos

Controle de Congestionamento Atual

O Estado da Arte: Redes modernas de alta velocidade e latência exigem algoritmos mais sofisticados que o TCP Reno clássico

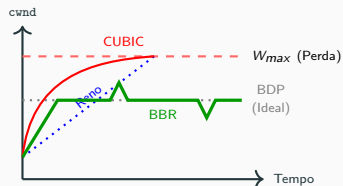
TCP CUBIC (Padrão Linux/Windows):

- **Crescimento:** Função cúbica
- **Ação:** Acelera rápido para recuperar a vazão e estabiliza suavemente perto do limite

TCP BBR (Google / GCP):

- **Paradigma:** Não usa perda de pacote. Estima atraso e banda continuamente
- **Vantagem:** Trabalha no ponto ideal (BDP), evitando lotar roteadores (*bufferbloat*)

Comparativo de Sondagem



Controle de Congestionamento: A Matemática do CUBIC

Independência de RTT: Diferente do Reno, o CUBIC calcula a Janela de Congestionamento (W) em função do **tempo** decorrido desde a última perda, garantindo justiça entre fluxos com latências diferentes

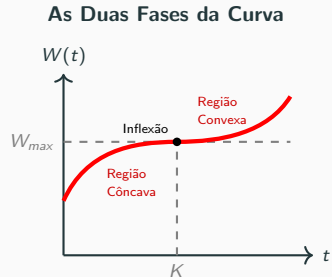
A Função Cúbica

$$W(t) = C \cdot (t - K)^3 + W_{max}$$

Onde:

- t : Tempo decorrido desde a última perda
- W_{max} : Tamanho da janela no momento da última perda
- C : Constante de escala do protocolo
- K : Tempo estimado para a janela retornar ao tamanho W_{max}

Redução mais Suave: Ao detectar perda, o CUBIC reduz a janela multiplicando-a por **0.7** (corte de 30%), diferente do corte drástico de 50% do Reno



Controle de Congestionamento: O Paradigma do BBR

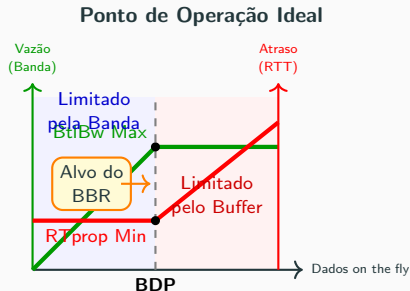
O Fim da Era das Perdas: O BBR não espera o buffer lotar. Ele constrói um modelo explícito do caminho, buscando operar no ponto onde a vazão é máxima e o atraso é mínimo (**BBR: Bottleneck Bandwidth and Round-trip propagation time**)

As Duas Variáveis do BBR:

- **BtlBw** (Banda do Gargalo): A taxa máxima de entrega que a rede suporta
- **RTprop** (Tempo de Propagação): O atraso mínimo da rota (quando não há filas)

O Ponto Ideal (BDP)

O BBR controla a taxa de envio para manter exatamente $BtlBw \times RTprop$ bytes *on the fly*



Evolução da Camada de Transporte

O Cenário Histórico: TCP e UDP dominam há 40 anos. Diferentes “sabores” de TCP foram desenvolvidos para lidar com cenários específicos:

Cenário	Desafios
Pipes longos e largos	Muitos pacotes <i>on the fly</i> ; a perda paralisa o fluxo
Redes sem fio	Perdas por ruído/mobilidade (TCP confunde com congestionamento)
Enlaces com longo atraso	RTTs extremamente altos
Data Centers	Altamente sensíveis à latência
Tráfego em background	Fluxos de baixa prioridade

A Nova Fronteira: Aplicação sobre UDP

A tendência atual é mover funções da camada de transporte para a **camada de aplicação**, por exemplo: o protocolo **QUIC** (usado no **HTTP/3**) implementa confiabilidade e controle de congestionamento rodando sobre o UDP

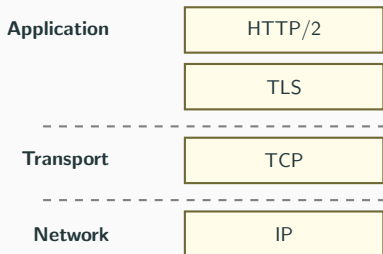
QUIC: Quick UDP Internet Connections

O Futuro do Transporte Web: O QUIC foi projetado pelo Google para resolver os gargalos de desempenho do TCP na web moderna

Características Principais:

- **Sobre o UDP:** É um protocolo implementado na camada de aplicação, que utiliza o **UDP** como base para a transferência
- **Objetivo Central:** Aumentar drasticamente o desempenho e reduzir a latência do tráfego HTTP
- **Adoção em Larga Escala:** Já está amplamente implantado em servidores do Google e em aplicativos como o navegador Chrome e o app móvel do YouTube

A Pilha Tradicional



Pilha clássica: HTTP/2 sobre TCP

QUIC: Conceitos Fundamentais

Quick UDP Internet Connections: O protocolo adota abordagens que estudamos neste capítulo para o estabelecimento de conexão, controle de erros e controle de congestionamento

1. Erros e Congestionamento

- Lógica herdada do TCP: utiliza algoritmos de detecção de perdas e congestionamento muito similares ao TCP clássico

2. Estabelecimento de Conexão

- Confiabilidade, controle de congestionamento, autenticação, criptografia e estado da conexão são estabelecidos em apenas **1 RTT**

3. Multiplexação de Streams

- Múltiplos *streams* (fluxos) em nível de aplicação são multiplexados sobre uma única conexão QUIC

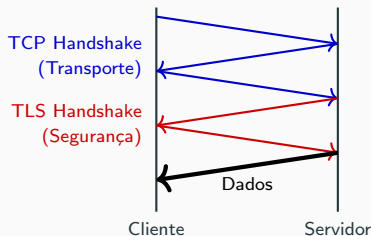
Gestão Interna da Conexão

- **Dados e Segurança:** A transferência confiável de dados e a segurança são tratadas de forma separada por *stream*
- **Congestionamento:** O controle de congestionamento é comum a toda a conexão QUIC

QUIC: Estabelecimento de Conexão

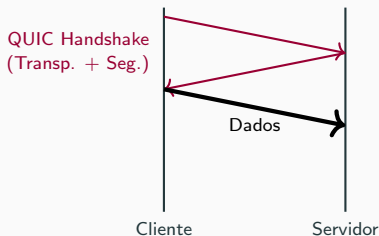
A Vantagem do 1-RTT: O QUIC unifica a negociação de transporte e segurança, reduzindo drasticamente a latência inicial

Pilha Clássica (TCP + TLS)



Custo: 2 Handshakes em série

A Abordagem QUIC



Custo: 1 Handshake consolidado

- **TCP:** O estado do transporte e da criptografia são negociados separadamente
- **QUIC:** Confiabilidade e criptografia são estabelecidas em conjunto

QUIC: Multiplexação e o Fim do HOL Blocking

O Problema do TCP: O *Head-of-Line (HOL) Blocking*. Como o TCP garante a entrega ordenada de **toda** a conexão, a perda de um único pacote paralisa todos os fluxos multiplexados

A Solução do QUIC (HTTP/3):

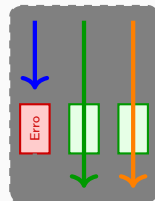
- O QUIC gerencia múltiplos *streams* (fluxos) de forma **independente** dentro da mesma conexão UDP
- **Criptografia e RDT isolados:** A transferência confiável de dados é feita por *stream*
- **O Resultado:** Se um pacote do “Fluxo A” for perdido, o “Fluxo B” e o “Fluxo C” continuam sendo entregues à aplicação sem atrasos

Impacto da Perda de Pacotes

TCP (HTTP/2) 1 Fluxo Global



QUIC (HTTP/3) Fluxos Isolados



QUIC: Padronização IETF

Do Experimental ao Padrão: O QUIC deixou de ser um protocolo proprietário do Google para se tornar o padrão oficial da Internet (RFCs publicadas em maio de 2021)

Documento	Descrição
RFC 9000	O protocolo de transporte base (Multiplexado e Seguro)
RFC 9001	Especificações de segurança e uso do TLS 1.3
RFC 9002	Algoritmos de detecção de perdas e controle de congestionamento
RFC 9114	HTTP/3: Protocolo de aplicação rodando sobre o QUIC
RFC 9221	Extensões para datagramas (transmissão não confiável)

- **IETF Working Group:** Mantém a evolução do protocolo com foco em interoperabilidade entre diferentes implementações
- **Uso em Larga Escala:** Google, Cloudflare e Facebook já utilizam estas RFCs para a maior parte de seu tráfego web moderno

QUIC: Recursos Avançados e Mobilidade

Além do Handshake: O QUIC introduz inovações que resolvem problemas crônicos de conectividade em dispositivos móveis

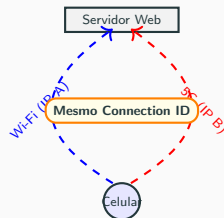
1. Migração de Conexão

- O TCP identifica conexões pelo IP e Porta. Se você muda do Wi-Fi para o 5G, a conexão cai
- **ID de Conexão:** O QUIC usa um identificador único. A conexão permanece ativa mesmo que o seu endereço IP mude

2. 0-RTT Resumption

- Em visitas subsequentes ao mesmo servidor, o cliente pode enviar dados no **primeiro pacote**
- Latência zero para o início da transferência de dados

Migração de Conexão na Prática



Resultado: Streaming de vídeo ou chamadas VoIP não sofrem interrupção ao trocar de rede

Panorama dos Protocolos de Transporte

Além do Básico: Embora TCP e UDP dominem, existem outros protocolos projetados para necessidades específicas de sinalização, mídia e mobilidade

Protocolo	Base	ID (IP)	Diferencial
TCP	IP	6	Confiabilidade e controle de fluxo
UDP	IP	17	Baixa latência, sem estado de conexão
SCTP	IP	132	Multi-homing e entrega por mensagens
DCCP	IP	33	UDP com controle de congestionamento
MPTCP	IP	6	Extensão para múltiplos caminhos (TCP)
QUIC	UDP	17	Handshake 1-RTT e segurança nativa

Observação para os Alunos

É interessante notar que o **SCTP** e o **DCCP** sofrem na Internet real por estarem “puros” sobre o IP: muitos roteadores e firewalls descartam esses pacotes por segurança ou ignorância. O **QUIC** aprendeu essa lição e “se vestiu” de **UDP** para passar despercebido e garantir interoperabilidade

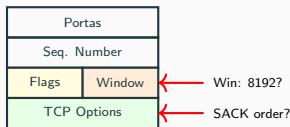
Transporte e Segurança: TCP Fingerprinting

A Identidade Oculta: Cada Sistema Operacional (SO) implementa a pilha TCP com sutis variações, permitindo a identificação remota do host

O que revela o SO?

- **TTL Inicial:** Geralmente 64 (Linux) ou 128 (Windows)
- **Janela (Window Size):** Tamanho inicial no pacote SYN
- **Opções TCP:** Ordem e presença de MSS, SACK, e *Window Scaling*

Análise de Digital TCP



Análise passiva: identifica o SO sem enviar pacotes extras

Uso em Segurança

- **Reconhecimento:** Ferramentas como `nmap` mapeiam o alvo passivamente
- **Deception:** Honeypots podem emular pilhas de sistemas vulneráveis para enganar atacantes

Resumo da Camada de Transporte

Serviços e Funcionalidades: A camada de transporte é a peça central que fornece comunicação lógica entre processos de aplicação em diferentes hosts

Principais Serviços:

- **Multiplexação:** Gerenciamento de múltiplos sockets através de portas
- **Confiabilidade (RDT):** Uso de somas de verificação, ACKs e retransmissões
- **Controle de Fluxo:** Mecanismo para não sobrecarregar o buffer do receptor
- **Controle de Congestionamento:** Regulação da taxa para proteger a rede

Protocolos de Internet:

- **UDP:** Simples, rápido e sem estado de conexão; ideal para tráfego tolerante a perdas
- **TCP:** Confiável, orientado à conexão e com controle de congestionamento
- **QUIC:** Combina a agilidade do UDP com segurança e controle de erros integrados

Conclusão do capítulo: Camada de transporte equilibra os requisitos das aplicações com as capacidades reais da rede